

Assignment #6
CE 303: Advanced Digital Logic Design

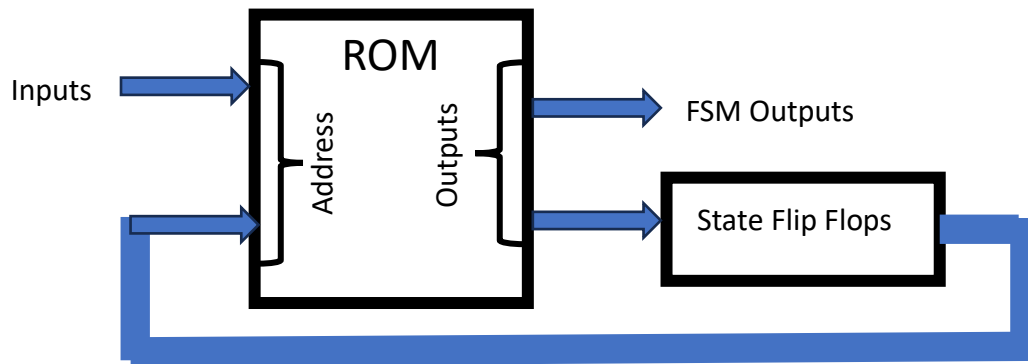
Problem 1. (20 Points)

Write a Verilog module Vredge for a state machine with one input X and one Moore-type output EDGE, which detects transitions on X. the machine tests its input X at each clock tick and asserts EDGE if the value of X at that tick is different from its value at the previous tick. Use state names A, B, C, and so on as needed. Simulate the FSM to demonstrate functionality. ALWAYS SUBMIT ALL VERILOG CODES.

Problem 2. (25 Points) Create an alternate implementation for the FSM from Problem 1 (Vredge module) using ONLY a ROM block and flip flops. Use the state encoding shown in the table below:

State Encoding
A 00
B 01
C 10
D 11

The general architecture is shown in the figure below:



- What is the size and word width configuration of the ROM for this implementation?
- Show the content of the ROM in a table formatted as shown below

ROM Address	ROM Outputs
00...0	
....	

- Draw the complete block diagram with all components and connections labeled with the signal names of the Vredge module.

Problem 3. (10 Points) FSM Design and Optimization

Reduce the number of states in the following state table and tabulate the reduced state table:

Present state	Next State		Output	
	X=0	X=1	X=0	X=1
A	F	B	0	0
B	D	C	0	0
C	F	E	0	0
D	G	A	1	0
E	D	C	0	0
F	F	B	1	1
G	G	H	0	1
H	G	A	1	0

Problem 4. (10 Points)

Assume that the propagation delay from clock to output of a D flip-flop is 5 ns and setup time is 5ns. What is the maximum clock frequency for the 4-bit ripple up counter discussed in Lecture 12 Slide 5?

Problem 5. (10 Points) Write a Verilog test bench that instantiates the two versions of a 3-bit counter with decoding in Programs 11-6 and 11-7, and runs them for a few dozen clock ticks. Examine the resulting output waveforms. Describe how they differ.

Program 11-6 Verilog code for a 3-bit counter and decoded outputs.

```
module Vr3bitctrdec ( CLK, CLR, S_L );
  input CLK, CLR;
  output reg [0:7] S_L;
  reg [2:0] Q;
  integer i;

  always @ (posedge CLK) // Create the counter f-f behavior
    if (CLR) Q <= 3'd0;
    else    Q <= Q + 1;

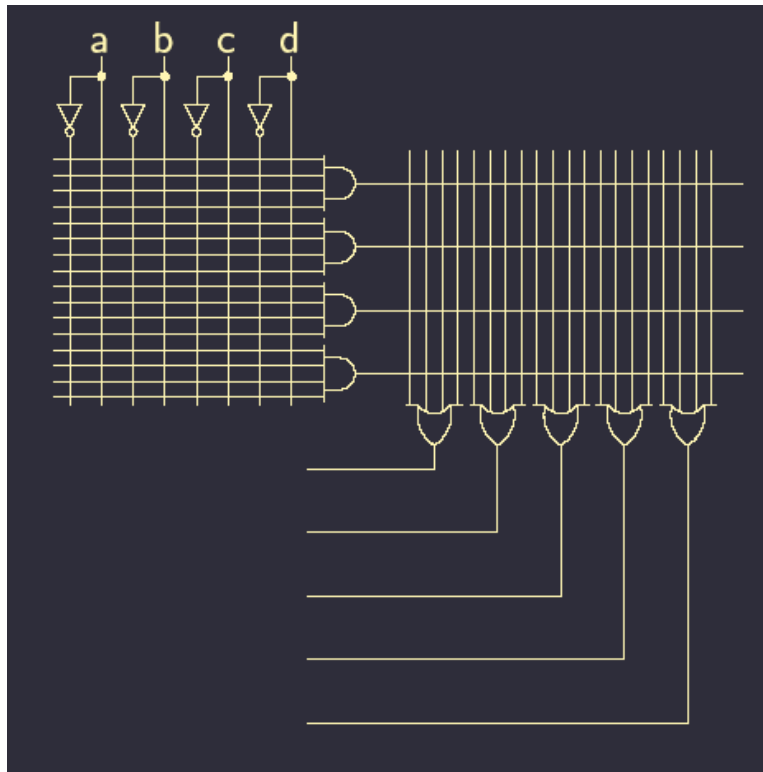
  always @ (Q) begin // Decode counter states to create outputs
    S_L = 8'b11111111;
    for (i=0; i<=7; i=i+1)
      if (i == Q) S_L[i] = 0;
    end
endmodule
```

Program 11-7 Verilog changes for a 3-bit counter module Vr3bitctrdecreg with registered, decoded outputs.

```
always @ (posedge CLK) begin
  if (CLR) Q <= 3'd0;    // Create the counter f-f behavior
  else   Q <= Q + 1;
  S_L <= 8'b11111111;    // Default for outputs is negated
  for (i=0; i<=7; i=i+1) // Decode counter states to assert
    if (i == Q) S_L[i] <= 0; // one active-low output
end
```

Problem 6. (15 Points) Revisit the minimized logic function you have identified in Assignment 1, Problem 3.

- What is the specifications of the smallest PLA that can implement this code scrambler? State the specs in the format of $N \times M \times K$, where N is the number of inputs (assume each input is made available in both non-inverted and inverted form), M is the number of AND gates, K is the number of OR gates.
- Show the connections using a diagram like the one shown below.



Problem 7 (10 Points)

Revisit Problem 2 of Assignment 5. Now consider clock skew, which is defined as the constant difference between the time of arrival of the same clock edge at the two consecutive Flip Flops in this circuit. Assume that there is clock skew between FF1 and FF2. The clock arrives at FF1 **by t_{skew} amount of time later** than at the FF2. How should the setup time and hold time constraints be re-written factoring in the clock skew? Write the new inequalities by inserting the skew parameter in the correct position.

